

OSAWARE — Architecture V2

Performance + Threading Audit (deferred work)

This document captures the Tier 2 and Tier 3 items from the BASIC compiler performance audit. Tier 1 cheap wins are being applied to `main` immediately; everything below requires more design work and is parked here for review.

Current state

Existing optimizations

- **OPT-E** — `_buildLineCache` pre-extracts each line as either `{entry, raw, paramStr}` (command) or `{assign, varName, rhs, exprNode}` (assignment). The tick fast-path executes cached assigns without re-parsing.
- **OPT-CACHE** — `_exprCache` Map (capped at 2000 entries) caches parsed expression and condition trees keyed by source string.
- **OPT-ASSIGN / OPT-COND** — both share the expression cache for assignment RHS and IF conditions.
- **OPT-F** — `evalCalc` fast paths: numeric literals, quoted strings, simple identifiers (≤ 12 chars).
- **OPT-H** — `lookup_` skips `String/trim/Number/switch` for plain variable reads.
- **Batched tick** — at `DELAY 0`, up to 100 000 statements OR 48 ms wall time per tick (Chrome flags any task > 50 ms as a long task).
- **rAF integration** — when GL just rendered, the next tick is scheduled via `requestAnimationFrame` so paints align to display refresh.

Threading model

- The `Kernel` (`core/kernel/kernel.js`) owns a process table keyed by PID; each process gets an isolated `ProcessMemory` (`core/kernel/process_memory.js`).
 - The scheduler currently runs only `activePid`. Other registered processes don't tick concurrently.
 - One JS event loop, one `setTimeout` / `rAF` chain, cooperative yielding.
-

Tier 2 — medium effort, larger payoff

6. Pre-tag every line as an opcode

Today, `_buildLineCache` only fast-paths simple assignments. Every command goes through `parseCode` at tick time, which walks `substring(0, 24..1)` against `_cmdMap`. Pre-tag each line at RUN time:

```
this._lineCache[ln] = { op: OP_PRINT, args: pre_extracted };
```

Then the tick inner loop becomes a `switch (cached.op)`. No string parsing at runtime.

Invalidation: any line write (`EDIT`, `RENUMBER`, source change) drops the cache (already done).

Expected payoff: 2-3× on text-heavy programs (PACMAN, AIDEMO). Mandelbrot/3D programs already dominate in `evalCalc`, so the gain there is smaller.

Risk: doubles the cache build cost at RUN. For 100 000-line programs this could be noticeable.

7. Constant folding in `_parseExprTree`

`4 * ATN(1)`, `PI * 2`, `1024 / 256` — fold at parse time. Trivial when both sides of a binary op are `{t:'n'}` nodes.

Expected payoff: small but free; saves the `_evalExprTree` recursion on these subtrees.

8. Type-specialized exec paths

Variables ending `$` are strings; others are numbers. In arithmetic-only ops (`OPER_PLUS` on two non-string operands), skip the `typeof` / NaN checks. The `_evalExprTree` switch on `OPER_PLUS` has an `ASS_ANY` branch with three runtime type tests — expensive on hot paths.

Expected payoff: 10-15 % on math-heavy code if the type can be proven at parse time.

9. Inline IF condition compilation

`IF cond THEN GOTO N` is one of the hottest forms. Currently parsed by `_parseCondTree` and stored in `_exprCache`. Could be folded into the line cache so the tick path is one opcode `OP_IF_GOTO` with a pre-parsed condition tree and a target line number.

Tier 3 — architectural

10. Per-SUB JIT to JS

When a SUB has been called more than N times (say 100), compile its body to a JS function via `new Function(varsCtx, ...)`. Cache the function on the SUB definition; subsequent CALLs invoke the JS directly. Math-heavy inner loops (MANDEL, MAZE3D physics, particle updates) potentially see 5-10× speedup.

Risks: - Source-line attribution for errors gets complicated; need a sourcemap from JS line → BASIC line. - `new Function` is blocked in some sandboxed environments; need a graceful fallback.

Design notes: - The compiled function takes a `ctx` object with `nums`, `strs`, `arrs`, `consts`. - All variable reads/writes go through the `ctx` — no closure capture of the Interpreter instance. - Calls from JIT back into runtime (e.g. `RND`, `INKEY`) use a small dispatch table also passed via `ctx`.

11. Real multitasking

Today the kernel schedules `activePid` only. To enable multiple BASIC programs sharing the CPU:

- Split the 48 ms tick budget across all `running` processes (round-robin or priority weighted).
- Each process gets `(48 / N_running)` ms. The batch loop checks `_processDeadline` instead of the global one.
- Background processes could tick in `requestIdleCallback` when no foreground tick is pending.

Use cases: an AIDEMO refresh in the background while the user types in another window; a long-running compute job that doesn't block input.

Risks: - Drivers (terminal, graphics) currently assume one active process per process-resource. Output multiplexing requires explicit ownership. - Some BASIC programs use `TIMER` for tight timing loops; their effective frame rate drops linearly with the number of running processes. Need a per-process `priority` knob.

12. Web Worker per process

True parallelism. Spawn a Worker for each BASIC process; the worker imports `compiler.js` + `kernel.js` and runs its own tick loop. Syscalls (terminal, graphics) marshal back to the main thread via `postMessage`.

Benefits: - A CPU-bound BASIC program (MANDEL) can't block the UI or other programs. - True multicore utilization on multi-process workloads.

Risks: - GL must stay on the main thread. The worker can't directly call Three.js; needs a synchronous-ish RPC that batches commands per frame. - BASIC's `_oop0bjects` shared model breaks; handles would need to be marshalled across workers (or just stay per-worker). - Memory cost: each worker is a separate JS context, with its own GC and code cache.

Recommended approach: start with Tier 3.11 (single-thread multitasking) before Tier 3.12 (workers). Validates the scheduling design without paying the marshalling cost.

Threading-safety considerations

These apply if either Tier 3.11 or 3.12 lands:

- **Drivers are not thread-safe:** `_gfxDrv`, `_glDrv`, `_audioDrv` hold mutable state. Either lock them to one process or refactor as syscall queues.
 - `window.focus()`, `document.fullscreen`, `setTimeout`: only the main thread can touch DOM/timers. Workers must marshal.
 - `_oop0bjects` **handle table**: process-local, no cross-process refs. Reference semantics break across workers.
 - `_exprCache` **is process-local**: already isolated by `ProcessMemory`. ✓
 - `_lineCache` **is process-local**: ✓
-

Risks across the board

- (6) Opcodes complicate EDIT/RENUMBER — invalidate cache on any line write (already done for assigns; extend to all opcodes).
 - (10) JIT makes line-number error reporting harder; need sourcemap.
 - (12) Workers break shared `_oop0bjects` model; would need handle marshalling or per-worker isolation.
 - All Tier 3 items make debugging harder — keep a `--no-opt` switch (`OPT OFF`) for triage.
-

This document is a deferred-work plan. Tier 1 cheap wins (keyword-lookup linearisation, hoisted toUpperCase, getAssignType memoization) are being applied to `main` now and don't appear here.